

Universitatea din București
Facultatea de Matematică și Informatică
Specializarea Informatică

LUCRARE DE LICENȚĂ

**Aplicație de analiză a codului și asistență în
generarea documentației proiectului: istoria
structurală a codului la nivel de commit-uri,
vizualizată drept graf interconectat**

Coordonator științific: Conf. dr. Boriga Radu

Absolvent: Alexandrescu Andra

București
iulie 2026

Rezumat

Aplicația reprezintă un instrument de analiză și explorare a codului sursă al unui proiect, care extrage funcții dintr-un proiect, acestea fiind observate la nivel de commit-uri, prin intermediul parserelor arborilor de sintaxă (*Abstract Syntax Tree*, AST [1]) specifice limbajelor TypeScript/ JavaScript, fiind observată structura codului, cât și evoluția temporală a conținutului. În cadrul acestei reprezentări, funcțiile extrase devin noduri într-un graf al proiectului, iar apelurile de funcții sunt definite prin muchii, reflectând astfel dependențele proiectului. De asemenea, aplicația combină un algoritm personalizat de PR (*PageRank* [2]) pentru extragerea clusterelor relevante unei interogări în limbaj natural, asupra grafului de cunoștințe, cu generarea unei documentații prin metoda semi-supervizată de învățare pe grafuri, DGI (*Deep Graph Infomax* [3]). PR propagă relevanța semantică de la cel mai similar nod din punct de vedere semantic cu interogarea, de-a lungul dependențelor structurale, iar DGI are rolul de a produce embedding-uri ale nodurilor. În final pe baza acestora, clusterizarea ierarhică aglomerativă (*Agglomerative Hierarchical Clustering* [4]), cu distanță cosinus și *average linkage*, grupează entitățile în cluster de nivel superior și subcluster, constituind harta structurală a codului sursă. Astfel, utilizatorul poate vizualiza graful de dependențe, alături de evoluția cronologică a codului, având posibilitatea de a alege seed-urile PR și a le integra în documentația structurală generată de DGI, obținând un draft editabil.

Abstract

The application is a tool designed to analyze and explore a project's source code by extracting functions and examining them at the level of Git commits, using AST (*Abstract Syntax Tree* [1]) parsers specific to TypeScript/ JavaScript, which enables to capture the code structure and its temporal evolution. Extracted functions become nodes in a project graph, while function calls are represented as edges, reflecting the project's dependencies. The application combines a custom PR (*PageRank* [2]) algorithm to extract relevant clusters for a natural language query on the knowledge graph, with semisupervised graph based documentation generation via DGI (*Deep Graph Infomax* [3]). PR propagates semantic relevance from the node that is most semantically similar to the query, along the structural dependencies, while DGI has the role of producing node embeddings. Based on these, agglomerative hierarchical clustering (*Agglomerative Hierarchical Clustering* [4]) (using cosine distance and *average linkage*) groups entities into higher level clusters and subclusters, constituting the structural map of the source code. Users can visualize the dependency graph and the code's chronological evolution, choose PR seed nodes and integrate them into the structural documentation generated by DGI, resulting in an editable draft.

Cuprins

Rezumat	1
Abstract	1
1 Introducere	4
1.1 Subiectul lucrării	4
1.2 Motivația și formularea problemei	4
1.3 Metoda propusă	4
2 Preliminarii	5
2.1 Noțiuni fundamentale	5
2.2 Stadiul actual al subdomeniului	5
3 Analiza și proiectarea sistemului	6
3.1 Interfață utilizator	6
3.2 Mediu de execuție desktop	7
3.3 Persistența datelor	7
3.4 Integrarea cu GitHub	7
3.5 Analiză statică a codului sursă	7
3.6 Învățare automată pe grafuri	8
4 Implementare	8
4.1 Analiza codului sursă și construirea grafului structural	8
4.1.1 Configurarea	8
4.1.2 Extragerea entităților, nodurilor, muchiilor și dependențelor	8
4.1.3 Post-procesare, vizualizare și persistență	9
4.2 Integrarea GitHub, sincronizarea commit-urilor și dimensiunea temporală a grafului	9
4.2.1 Credențiale GitHub și înregistrarea webhook-ului	10
4.2.2 Evenimente webhook, sincronizarea commit-urilor și actualizarea folder-ului local	10
4.2.3 Analiza temporală a funcțiilor și integrarea istoricului în graf	11
4.3 Procesarea semantică: embedding-uri, DGI, căutare PR și generarea documentației	11
4.3.1 Arhitectura generală	11
4.3.2 Pre-procesarea grafului și generarea embedding-urilor	12
4.3.3 Antrenarea reprezentărilor DGI	13

4.3.3.1	Etapa nesupervizată	13
4.3.3.2	Etapa generării setului de perechi interogare-nod	13
4.3.3.3	Etapa semisupervizată	14
4.3.4	Căutarea semantică de clustere prin PR	15
4.3.5	Generarea documentației prin DGI	16
4.3.6	Evaluarea metodelor abordate	17
4.3.6.1	Evaluarea recuperării de noduri în mod intrinsec	17
4.3.6.2	Evaluarea recuperării de noduri în mod extrinsec, condiționat de interogări	17
4.3.6.3	Evaluarea clusterizării	19
4.4	Partajarea proiectelor între utilizatori	19
4.4.1	Trimiterea și primirea pachetelor de date	19
4.4.2	Roluri, acces și notificări	20
5	Concluzie	20
	Bibliografie	21
	Anexă 1: Rezultatele evaluărilor experimentale	24
	Evaluarea recuperării de noduri în mod intrinsec	24
	Evaluarea recuperării de noduri în mod extrinsec	25
	Evaluarea clusterizării	27
	Anexă 2: Interfața aplicației	28

1 Introducere

1.1 Subiectul lucrării

Lucrarea propusă spre susținere pune accent pe inginerie software și metode de inteligență artificială aplicate codului sursă. Astfel, lucrarea aparține subdomeniilor instrumente pentru dezvoltatori (*developer tools*), grafuri de cod (*code knowledge graphs*), învățare automată pe grafuri (*graph representation learning*).

Lucrarea abordează problema generării automate a unui graf de cunoștințe derivat din analiza codului sursă și a dependențelor dintre entitățile extrase, proximitatea semantică a acestora și evoluția lor temporală. Scopul este oferirea unei reprezentări, care este structurală, semantică și temporală, facilitând înțelegerea și navigarea unei baze de cod (codebase) de către o echipă de dezvoltare a proiectelor software.

1.2 Motivația și formularea problemei

În acest sens, motivația temei evidențiază necesitatea analizei evoluției unui proiect software, care nu poate fi redusă la o succesiune de commit-uri Git, deoarece commit-urile descriu modificări în contextul controlului de versiuni (*version control*), dar nu reflectă structura logică a codului. Astfel, structura logică este definită prin dependențele structurale și semantice dintre funcții și module, fiind oferită o perspectivă esențială pentru organizarea și colaborarea unei echipe de dezvoltare. În plus, este necesară și o abordare semantică, prin soluții de căutare semantică la nivel de cod. De aceea, tema este relevantă: unificarea celor trei perspective într-un singur instrument poate reduce fragmentarea fluxurilor de lucru ale dezvoltatorilor.

1.3 Metoda propusă

Așadar, soluția propusă unifică cele trei niveluri într-un cadru coerent, în loc să le trateze ca module independente. Astfel, codul sursă este transformat într-un graf de cunoștințe prin analiză statică bazată pe parsere AST (*Abstract Syntax Tree* [1]), unde nodurile extrase reprezintă entități de tipul funcțiilor și metodelor, iar muchiile descriu dependențe statice, precum apeluri directe, referințe și importuri. Acest strat structural este completat de un strat temporal, obținut din istoricul Git, unde, pentru fiecare commit, se analizează fișierele adăugate, modificate sau șterse și se urmărește evoluția entităților în timp. Spre deosebire de abordările care atașează separat un motor de căutare semantică [5], înțelegerea semantică este integrată direct în graf prin reprezentări învățate: metoda semisupervizată DGI (*Deep Graph Infomax* [3]), bazată pe o rețea convoluțională pe grafuri (*Graph Convolutional Network*, GCN), produce embedding-uri care surprind atât proximitatea structurală,

cât și coerența funcțională a nodurilor. Aceste reprezentări permit atât interogări în limbaj natural prin propagarea relevanței în graf cu un algoritm PR (*PageRank* [2]), cât și clusterizarea automată a subgrafurilor tematice, realizată prin clusterizare ierarhică aglomerativă (*Agglomerative Hierarchical Clustering* [4]) pe embedding-urile DGI. Rezultatul este o sugestie structurată de documentație: grupări funcționale identificate algoritmic, bazate pe relațiile dintre entități și evoluția lor temporală.

2 Preliminarii

2.1 Noțiuni fundamentale

Conceptele abordate în această lucrare sunt analiza statică a codului, reprezentarea grafică a dependențelor, căutarea semantică și învățarea pe grafuri. În cele ce urmează sunt prezentate noțiunile teoretice care stau la baza soluției propuse.

Arborele de sintaxă (*Abstract Syntax Tree*, AST [1]) este o reprezentare ierarhică a structurii sintactice a unui program, în care nodurile reprezintă construcții de cod, iar muchiile relațiile dintre ele.

Căutarea semantică a codului (*Semantic Code Search* [5]) identifică fragmente relevante pe baza semnificației, nu a unei potriviri textuale.

PageRank (PR) [2] ordonează nodurile unui graf orientat modelând un utilizator care urmează muchiile cu o anumită probabilitate și sare la un nod ales la întâmplare (*teleportare*).

O rețea convoluțională pe grafuri (*Graph Convolutional Network*, GCN [6]) actualizează reprezentarea fiecărui nod prin agregarea informației de la vecinii săi. *Deep Graph Infomax* (DGI) [3] este o metodă nesupervizată de învățare pe grafuri care antrenează un encoder GCN astfel încât fiecare nod să fie aliniat cu un sumar global al grafului, discriminatorul distingând perechile reale de cele corupte. În lucrare, DGI este extins semisupervizat pe perechi nod-limbaj natural.

Clusterizarea ierarhică aglomerativă (*Agglomerative Hierarchical Clustering* [4]) fuzionează iterativ cele mai apropiate clustere, pornind de la noduri individuale (*bottom-up*).

2.2 Stadiul actual al subdomeniului

Abordările existente acoperă, în general, doar două dintre cele trei niveluri de abstractizare relevante pentru înțelegerea unui proiect software: cel *temporal* (istoricul modificărilor) și cel *structural* (dependențele dintre entități). Nivelul *semantic* este tratat de regulă separat, prin soluții de căutare semantică a codului (*Semantic Code Search*) [5], care combină LLM-uri și embedding-uri vectoriale. Modele precum **CodeBERT** [7] și **UniXcoder** [8], antrenate pe cod sursă, obțin rezul-

tate bune la similitudinea între fragmente de cod, dar alinierea cod, limbaj natural rămâne dificilă.

Platformele comerciale extind această direcție spre agenți și interogări în limbaj natural: **Sourcegraph** [9] oferă căutare semantică la scară de codebase și context pentru agenți prin MCP, **Sonde** [10] construiește un graf structural cu integrare **Git** și analiză temporală, iar **Pieces Desktop** [11] oferă memorie pe termen lung a fluxului de lucru și căutare conversațională, tot prin MCP. În literatura de cercetare, au fost propuse și metode care integrează căutarea semantică cu grafuri de apeluri și GCN [12], însă fără o abordare care să includă integrarea Git sau generarea documentației unui proiect.

Reprezentările pe grafuri (*Node2Vec*, *GNN*, *DGI*) îmbunătățesc modelarea relațiilor dintre entități, dar puține soluții software le combină cu sincronizarea **Git**, explorarea istoricului codului și un flux coerent de documentare automată. Așadar, lucrarea de față unifică nivelul temporal, structural și semantic într-o aplicație.

3 Analiza și proiectarea sistemului

Alegerea acestor tehnologii este determinată de caracterul *local-first* [13] al aplicației și al necesității accesului controlat la sistemul de fișiere. Astfel, cerința de a integra procese de analiză software cu metode de învățare automată necesită stocarea pe calculatorul utilizatorului. Limbajele de programare folosite sunt **TypeScript**, rulat pe **Node.js** (pentru interfață, API, orchestrare desktop și analiză statică a codului sursă) și **Python** (pentru componentele de învățare automată: preprocesarea grafurilor, embedding-uri **UniXcoder**, antrenarea și inferența *DGI*, clusterizarea prin *PageRank* personalizat și generarea documentației structurale).

3.1 Interfață utilizator

Partea de frontend este construită cu framework-urile **Next.js** și **React**, în timp ce stilizarea este realizată cu **Tailwind CSS**.

În plus, componentele interfeței precum formularele de autentificare, înregistrare, creare proiect și configurare credențiale GitHub sunt gestionate prin **React Hook Form**, cu validare declarativă definită în scheme **Zod 4**.

Vizualizarea grafului de cunoștințe se bazează pe biblioteca **reagraph** [14], a cărei randare se face în **WebGL**, nu prin SVG sau DOM, nodurile și muchiile fiind vizualizate eficient pe un canvas, ceea ce permite navigarea fluentă în grafuri cu multe entități. Există limitări, fiind suficiente operații pe GPU care depind de complexitatea grafului proiectului, motiv pentru care încărcarea vizualizării și a interacțiunilor în canvas este influențată direct. În plus, afișarea codului sursă din commit-uri în mod cronologic se bazează pe **react-syntax-highlighter**, care aplică evidențiere

(*syntax highlighting*) prin motorul dedicat evidențierii sintactice numit Prism, inferând limbajul de programare automat din extensia fișierului.

3.2 Mediu de execuție desktop

Shell-ul aplicației este realizat cu **Electron**, care conține interfața web și oferă acces la operațiile native necesare unei aplicații de analiză a codului: selectarea directoarelor de proiect pentru citire și scriere, lansarea proceselor de analiză și gestionarea datelor locale.

La acest nivel sunt prezente servicii auxiliare, iar acestea au rolul de a comunica cu **GitHub**, de aceea sunt folosite platformele **cloudflared** pentru expunerea temporară a serviciilor locale către internet și **Azure Functions** care primește webhook-uri **GitHub**.

3.3 Persistența datelor

De asemenea, deși aplicația urmează un model *local-first*, este necesar un strat de coordonare via **PocketBase**, care rulează local și este un backend ușor, cu o bază de date **SQLite** încorporată. **PocketBase** gestionează conturile utilizatorilor și sesiunile, înregistrările de proiect, credențialele **GitHub** (token, secret webhook), evenimentele recepționate de la **GitHub** și partajările între utilizatori. Aceste date trebuie să fie accesibile atât API-ului local, cât și serviciului **Azure Functions** (prin tunel), spre deosebire de fișierele de dimensiuni mari, care rămân stocate local, exemplu fiind grafurile, embedding-urile, snapshot-urile de commit-uri și documentațiile care se află în folder-ul de sistem AppData.

3.4 Integrarea cu GitHub

Integrarea cu **GitHub** este realizată prin **GitHub REST API** și **Azure Functions**. Această componentă permite aplicației să primească evenimente de tip **git push**, să valideze autenticitatea acestora și să sincronizeze informațiile relevante despre commit-uri.

Pe lângă integrarea prin API, sistemul utilizează și **Git** local pentru operații de tip **git pull** în directorul proiectului. Astfel, este posibilă menținerea unei copii locale actualizate a codebase-ului și facilitează corelarea modificărilor **Git** cu entitățile extrase prin analiza statică.

3.5 Analiză statică a codului sursă

Extragerea entităților funcționale și a dependențelor structurale se realizează prin parsarea codului sursă TypeScript și JavaScript cu **ts-morph** [15], bibliotecă care oferă acces programatic la AST, fără a utiliza language server-ul TypeScript/ JavaScript **tsserver**, ci constituie un strat de abstracție peste TypeScript Compiler API. Alegerea acestei soluții este motivată de faptul că analiza abordată

în această lucrare se referă la proiectele dezvoltate în limbajele TypeScript și JavaScript. O extindere către mai multe limbaje de programare ar impune fie adoptarea unui language server dedicat fiecărui limbaj, fie implementarea manuală a mecanismelor de rezolvare a apelurilor între fișiere (*cross-file calls*), adaptate la particularitățile fiecărui parser și modulele fiecărui limbaj. De aceea, analiza statică a codului este limitată la TypeScript/ JavaScript.

3.6 Învățare automată pe grafuri

Reprezentările semantice și operațiile de clusterizare sunt implementate în Python, folosind **PyTorch** [16] și **PyTorch Geometric** [17] pentru reprezentarea grafului și antrenarea encoderului *GCN*, *Deep Graph Infomax* (DGI) pentru embedding-uri sensibile la topologie, **UniXcoder** (prin Hugging Face Transformers) pentru codificarea inițială a nodurilor, **scikit-learn** [18] pentru clusterizare ierarhică aglomerativă și similaritate cosinus, și *PageRank personalizat* pentru identificarea subgrafurilor relevante unei interogări în limbaj natural.

4 Implementare

4.1 Analiza codului sursă și construirea grafului structural

4.1.1 Configurarea

În contextul analizei unui proiect, este necesară selectarea de către utilizator a folder-ului care conține codul sursă al repository-ului local. Astfel, selectarea căii via mecanismul IPC al shell-ului Electron implică o asociere între utilizator și calea proiectului stocată într-un fișier de setări al aplicației, astfel încât alte analize ulterioare și sincronizarea commit-urilor să poată fi realizate fără reselectarea folderului.

4.1.2 Extragerea entităților, nodurilor, muchiilor și dependențelor

În continuare, parcurgerea codului sursă este realizată recursiv din calea proiectului, iar acest fapt presupune identificarea tuturor fișierelor TypeScript și JavaScript, fiind excluse folderele de dependențe, build, control de versiune și medii virtuale. Astfel, fiecare fișier este transformat într-un arbore *Abstract Syntax Tree* [1], iar din acesta se extrag entități și relații între acestea.

Din arborii sintactici obținuți se extrag, în cadrul aceleiași analize, atât nodurile grafului, cât și muchiile dintre ele. Nodurile corespund funcțiilor de nivel superior (*export function*), metodelor din clase și expresiilor funcționale (*arrow functions* sau funcții anonime atribuite variabilelor), iar pentru fiecare astfel de entitate se rețin identificatorul unic derivat din calea și poziția din fișier a

funcției, tipul și numele, cât și conținutul funcției, deoarece aceste informații sunt utilizate ulterior la generarea embedding-urilor semantice.

Referitor la relațiile care se stabilesc între entități, muchiile exprimă dependențe statice din cod, fără executarea acestuia. Astfel, este parcurs corpul fiecărei entități și sunt identificate relațiile cu alte funcții. Există două categorii de muchii extrase, deoarece este necesară o diferență de abordare pentru stabilirea relațiilor între entități. Muchiile de tip apel (**CALLS**) corespund invocărilor directe ale funcțiilor, fie în același fișier, fie între fișiere diferite, precum și utilizării componentelor React în JSX, deoarece acestea sunt interpretate ca invocări ale funcțiilor care definesc componentele respective. Muchiile de tip utilizare (**USES**) surprind referințe la alte funcții fără apel imediat, cum ar fi transmiterea unui callback ca argument sau folosirea unei funcții ca middleware. În aceeași categorie intră și dependențele din fișierele care importă module prin *require*, deoarece legătura se stabilește prin utilizarea numelui importat în cod, nu printr-o expresie de apel cu paranteze, iar rezolvarea simbolică TypeScript dată de librăria *ts-morph* omite acest caz.

4.1.3 Post-procesare, vizualizare și persistență

Graful brut obținut după inferența muchiilor poate conține noduri fără nicio legătură cu restul rețelei, acest fapt însemnând că există funcții extrase din *AST* niciodată apelate sau referențiate. Aceste noduri izolate sunt eliminate în etapa de post-procesare, deoarece nu contribuie la înțelegerea structurii dependențelor.

În interfață, graful este transformat într-o structură de vizualizare prin intermediul **reagraph** și grupează nodurile după fișier, separă muchiile în apeluri *in-file* și *cross-file* și calculează componentele conexe posibile. În plus, partiționarea în componente conexe este o măsură de gestionare a complexității, însemnând că în mod implicit, interfața afișează o singură componentă, permițând utilizatorului să navigheze între subgrafuri disjuncte ale codebase-ului, astfel evitând supraîncărcarea randării la scară mare.

Fișierul final conține graful structural complet, statisticile analizei și metadatele de vizualizare, împreună cu calea rădăcină a proiectului analizat.

Interfața canvasului care conține graful de cunoștințe este ilustrată în Figura 2.1 din Anexa 2.

4.2 Integrarea GitHub, sincronizarea commit-urilor și dimensiunea temporală a grafului

Interfața paginii dedicate commit-urilor GitHub, folosită pentru inspectarea evenimentelor primite de aplicație de la GitHub, este prezentată în Figura 2.2 din Anexa 2.

4.2.1 Credențialele GitHub și înregistrarea webhook-ului

Conectarea la **GitHub** se realizează din interfață, unde utilizatorul care deține proiectul completează un formular cu tokenul **GitHub** de acces personal, numele owner-ului și al repository-ului. Astfel, datele sunt validate și transmise către API-ul aplicației, care le stochează în **PocketBase** sub forma unei înregistrări de credențiale asociate utilizatorului și proiectului.

Folosind token-ul utilizatorului, serverul creează pe **GitHub** un webhook care verifică evenimentele de tip *push* și *pull request*, direcționate către funcția **Azure**. Deoarece funcția care primește evenimente GitHub, existentă în **Azure Functions**, este expusă public, server-ul generează un secret de webhook și îl salvează împreună cu token-ul în **PocketBase**, precum și în configurația webhook-ului de pe GitHub, pentru a putea verifica autenticitatea evenimentelor primite. Astfel, de fiecare dată când are loc o acțiune asupra repository-ului, GitHub semnează payload-ul cu acest secret, iar funcția Azure validează semnătura înainte de procesare, astfel încât doar evenimentele autorizate sunt acceptate.

După validare, funcția identifică proiectul și utilizatorul după owner-ul și numele repository-ului, consultând credențialele stocate în PocketBase, apoi redirecționează evenimentul către API-ul local al aplicației, accesibil prin tunelul **cloudflared**. În cazul în care validarea repository-ului sau crearea webhook-ului eșuează, înregistrarea credențialelor este anulată, astfel încât proiectul să nu rămână într-o stare parțial configurată.

4.2.2 Evenimente webhook, sincronizarea commit-urilor și actualizarea folder-ului local

În continuare, la primirea unui eveniment de tip *push*, backend-ul normalizează payload-ul în înregistrări individuale de tip commit și le salvează într-o colecție de evenimente webhook, evitând duplicatele pentru același SHA. Pentru fiecare commit nou, serverul interoghează API-ul GitHub pentru lista fișierelor adăugate, modificate sau șterse, apoi salvează conținutul local, împreună cu un fișier numit **manifest.json** care reține metadate despre conținutul fiecărui commit.

În interfață, commit-urile sunt listate din evenimentele webhook stocate în **PocketBase**. Sincronizarea suplimentară cu GitHub, care este declanșată la deschiderea proiectului sau la solicitarea explicită a utilizatorului, are rolul de recuperare și completare a istoricului. În general acest fapt este necesar în două situații: la prima configurare a credențialelor, când istoricul preexistent trebuie importat din GitHub, și la repornirea aplicației, când commit-urile produse în absența tunelului **cloudflared** sau a funcției **Azure** nu au putut fi receptate în timp real, atunci când aplicația nu rulează. Așadar, înregistrările din PocketBase sunt completate în final, iar fișierele lipsă sunt exportate în folder-ul de commit-uri din **AppData**.

În timpul rulării aplicației, commit-urile sunt receptate prin webhook-uri, ce presupune că fiecare eveniment este înregistrat în PocketBase, iar fișierele asociate sunt exportate local. În plus,

se realizează o verificare periodică cu GitHub atunci când un eveniment nu a fost livrat, însemnând că webhook-ul nu a funcționat temporar chiar dacă aplicația rulează. În cazul în care astfel de commit-uri noi sunt detectate în timp ce utilizatorul vizualizează proiectul, interfața solicită actualizarea folder-ului sursă al proiectului printr-un *git pull* executat de shell-ul Electron, astfel încât analiza structurală ulterioară să reflecte codul actualizat alături de evenimentele webhook recepționate.

4.2.3 Analiza temporală a funcțiilor și integrarea istoricului în graf

Analiza codului sursă este realizată din interfața grafului de cunoștințe prin IPC-ul shell-ului Electron și este executat script-ul de analiză cu extragerea entităților asupra folder-ului proiectului configurat.

În cadrul aceleiași execuții, script-ul realizează mai întâi analiza structurală a proiectului întreg, descrisă în secțiunea anterioară și salvează rezultatul. Ulterior, dacă există directoare de commit-uri în **AppData**, acestea sunt procesate cronologic, iar pentru fiecare commit se parsează doar fișierele adăugate sau modificate. Urmează ca din fișierele parsate să fie extrase funcțiile, iar pentru a stabili modificările asupra fiecărei funcții, script-ul le compară cu lista completă a funcțiilor întâlnite în commit-urile procesate anterior, marcându-le ca adăugate, modificate sau șterse. La final, toate modificările înregistrate per commit sunt combinate într-un fișier care reține istoricul fiecărei funcții.

Astfel, în cadrul vizualizării grafului de cunoștințe, istoricul fiecărei funcții este adăugat ca metadată, fără a modifica muchiile sau topologia grafului. Așadar, nodurile își schimbă culoarea în funcție de ultima modificare înregistrată (**added**, **modified**, **removed**), iar la selectarea unui nod utilizatorul poate consulta istoricul commit-urilor asociate. În plus, un subgraf separat ilustrează funcțiile șterse definitiv din codebase, completând evoluția istorică a proiectului.

4.3 Procesarea semantică: embedding-uri, DGI, căutare PR și generarea documentației

4.3.1 Arhitectura generală

Procesarea semantică a grafului de cunoștințe este organizată în diferite etape, fiind folosită arhitectura modelului *Deep Graph Infomax* [3]. Stratul de pre-procesare produce, pentru fiecare proiect, un fișier care îmbină vectorii **UniXcoder** ai nodurilor cu structura **PyTorch Geometric** a grafului, iar acest strat este necesar atât pentru antrenarea modelelor, cât și pentru inferența din interfață. Stratul de antrenare, executat în afara fluxului din aplicație, antrenează encoderul *DGI* în mod ne-supervizat pe un corpus limitat de proiecte, apoi îl antrenează suplimentar supervizat pe perechi formate din interogări și noduri. Stratul de inferență este expus în interfața grafului de cunoștințe

și folosește două reprezentări diferite: căutarea de clustere de noduri prin *PR*, care operează în spațiul brut **UniXcoder**, cu proiecția interogărilor în limbaj natural și generarea documentației, care utilizează embedding-urile rafinate *DGI*.

4.3.2 Pre-procesarea grafului și generarea embedding-urilor

Înainte de orice operație semantică, frontend-ul verifică prin endpoint-ul corespunzător dacă graful proiectului conține embedding-urile necesare etapelor de antrenare sau inferență, iar în cazul în care fișierul ce conține preprocesarea grafului unui proiect nu există, server-ul execută un script, care calculează embedding-urile **UniXcoder** pentru fiecare funcție, le cache-uește și construiește structura **PyTorch Geometric** a grafului, salvând rezultatul final.

Așadar, script-ul citește fișierul de analiză *AST* a grafului și încarcă modelul **UniXcoder** (**microsoft/unixcoder-base** [8]), ales în locul **CodeBERT** [7], deoarece separă mai bine conținutul funcțiilor distincte în spațiul vectorial. Într-un stadiu anterior al lucrării, s-a comparat distribuția similarității cosinus între perechi aleatoare de funcții din același proiect, pentru **CodeBERT** și **UniXcoder**. **CodeBERT** a produs valori medii mult mai ridicate, însemnând că embedding-urile funcțiilor diferite erau frecvent foarte apropiate în spațiul vectorial. În acest caz, pentru un graf de cunoștințe, căutarea semantică nu ar descoperi noduri relevante, deoarece funcțiile distincte primesc reprezentări aproape identice. **UniXcoder** a obținut o similaritate medie mai scăzută între perechi oarecare, indicând un spațiu mai bine separat. Dintr-o perspectivă teoretică, **CodeBERT** este antrenat prin masked language modeling pe cod sursă, fără aliniere explicită între limbaj natural și cod, în timp ce **UniXcoder** folosește pre-antrenare multimodală, incluzând perechi cod-comentariu și învățare contrastivă, astfel încât reprezentările să capteze atât semantica funcțională, cât și structura sintactică.

În continuare, pentru fiecare nod, codul sursă al funcției este encodat într-un vector de 768 de dimensiuni. Această valoare corespunde dimensiunii de ieșire a arhitecturii folosite de **UniXcoder**. Deoarece modelul **UniXcoder** acceptă o intrare limitată la 512 de tokeni, funcțiile ale căror cod sursă depășește această lungime sunt segmentate cu o suprapunere de 128 de tokeni între segmente adiacente, pentru a evita pierderea contextului. Astfel, fiecare segment produce un vector de reprezentare, iar embedding-ul final al funcției se obține prin medierea acestor vectori parțiali. Rezultatele sunt memorate în cache, sub formă de fișiere **.npy**, în contextul evitării recalculării la rulări ulterioare ale pre-procesării. Graful este transformat într-un graf neorientat deoarece relația dintre muchii nu oferă informație relevantă la nivel de învățare automată. Într-un final, rezultatul final este stocat local, cu extensia **.pkl**, conținând matricea de caracteristici, lista nodurilor cu muchiile neorientate și embedding-urile funcțiilor.

4.3.3 Antrenarea reprezentărilor DGI

Antrenarea modelelor nu este declanșată din interfața utilizatorului, ci constituie o fază de pregătire a corpus-ului. Antrenarea a fost realizată pe un corpus limitat de 67 de proiecte, cu un număr de 11420 fișiere analizate, având un total de 25935 noduri și 136129 de muchii identificate. Inițial, abordarea a fost transductivă, care ar fi cerut reantrenare la fiecare proiect nou, dar în implementarea finală este antrenat un encoder de graf inductiv, ce presupune o funcție cu ponderi învățate pe corpus-ul de antrenare și refolosite la inferență.

În vederea asocierii interogărilor de limbaj natural cu reprezentările nodurilor, pipeline-ul antrenează două encodere *DGI*, mai exact unul nesupervizat, care învață structura grafurilor fără interogări etichetate, și unul semisupervizat, care aliniaza embedding-urile nodurilor cu interogări.

4.3.3.1 Etapa nesupervizată

Encoderul nesupervizat *DGI* primește embedding-uri **UniXcoder** de 768 dimensiuni, le proiectează într-un spațiu ascuns de 512, apoi produce reprezentări finale tot de 512 dimensiuni, prin trei straturi de agregare mean în stil *GraphSAGE* [19]: la fiecare strat, reprezentarea unui nod se construiește din media vecinilor săi direcți, urmată de o transformare liniară. Nu este o implementare universală *GraphSAGE*, deoarece nu are eșantionare de vecini și nu este folosită convoluția *SAGEConv* din PyG, însă presupune faptul că informația nodului curent este reintrodusă între straturi prin conexiuni reziduale (skip connections), astfel încât fiecare nivel combină contextul local cu reprezentarea inițială a nodului.

În plus, antrenarea folosește o schemă contrastivă de tip *DGI* în care un discriminator bilinear învață să distingă perechile valide (formate din reprezentarea nodului împreună cu sumar global al grafului) de perechile false. În formularea originală a metodei, exemplele negative sunt generate în interiorul aceluiași graf, prin amestecarea caracteristicilor nodurilor, păstrând structura muchiilor neschimbată. În implementarea abordată, *DGI* este adaptat pentru antrenare multi-graf, adică negativele sunt generate diferit atunci când batch-ul are mai multe proiecte. În practică, nodurile fiecărui graf sunt comparate cu sumarul global al unui alt proiect din același batch, nu cu o versiune coruptă a aceluiași graf. Dacă batch-ul conține un singur graf, se revine la mecanismul clasic de corupție prin permutarea caracteristicilor. Această adaptare păstrează scopul de a maximiza *mutual information*, deci o face compatibilă cu antrenarea inductivă pe mai multe proiecte simultan.

4.3.3.2 Etapa generării setului de perechi interogare-nod

Înainte de a descrie procesul de antrenare semisupervizată, este necesar a menționa metoda de determinare a setului de interogări formulate în limbaj natural, prin intermediul căreia se realizează alinierea dintre spațiul grafurilor și cel al reprezentărilor semantice.

Perechile interogare-nod nu sunt etichetate manual, ci generate prin sumarizare a codului sursă

folosind **CodeT5** [20], cu selecție după similaritatea față de embedding-urile **UniXcoder** ale nodurilor țintă.

Perechile rezultate sunt într-un număr redus și împărțite stratificat după gradul nodului țintă în seturi de antrenare/ validare/ testare, iar pe întregul corpus de 67 de proiecte rezultă 8347 perechi de antrenare, 1042 de validare și 1076 de test (10465 în total). Consistența acestor date este importantă deoarece evaluarea și antrenarea semisupervizată presupun că interogările rămân aliniate la noduri reale din graful curent și că split-urile train/ validation/ test au dificultate comparabilă, altfel metricile de pe antrenare nu reflectă performanța pe validare sau pe proiecte noi. În acest context, perechile sunt împărțite în train/ validation/ test în mod stratificat după gradul nodului țintă, în trei bucket-uri: low (≤ 5), medium (6–15), high (≥ 15).

În varianta inițială, fără stratificare și fără verificarea mapării între interogare și nod, split-urile puteau fi dezechilibrate. De exemplu, antrenarea conținea preponderent noduri cu un grad ridicat de conectivitate, deci o preferință a modelului către funcții centrale, iar validarea/ testarea conțineau noduri izolate. Acest fapt ar fi însemnat că antrenarea și evaluarea se fac pe distribuții diferite de dificultate (*distribution shift*), deci consecința era o supraestimare a performanței pe antrenare și o scădere clară la validare/ testare.

4.3.3.3 Etapa semisupervizată

În continuare, etapa semisupervizată aliniază reprezentările structurale ale nodurilor cu interogări în limbaj natural și produce encoderul folosit la generarea documentației și la evaluarea *DGI*. Spre deosebire de antrenarea nesupervizată, care maximizează informația mutuală prin discriminatorul bi-liniar din *DGI*, etapa semisupervizată nu reutilizează greutatele determinate anterior, ci inițializează un encoder cu aceeași arhitectură (trei straturi mean în stil *GraphSAGE*, cu conexiuni reziduale), dar cu un spațiu ascuns de 256 de dimensiuni și un spațiu al reprezentărilor finale de 768 de dimensiuni.

Înainte de finetuning, este antrenată o proiecție liniară peste **UniXcoder**, care mapează interogările în spațiul de 768 de dimensiuni al embedding-urilor brute ale nodurilor, folosind aceeași pierdere contrastivă *InfoNCE* cu negative în lot [21, 22]. Rezultatul este necesar pentru inferența *PR* și inițializează alinierea cu limbajul natural la finetuning.

La finetuning, corpus-ul este împărțit la nivel de proiect, mai exact aproximativ 40% din proiecte (26 din 67) au fost rezervate integral pentru validare, iar toate perechile interogare-nod din aceste proiecte au fost excluse din antrenare, astfel încât validarea e inductivă la nivel de graf.

Deși atât proiecția interogărilor, cât și ieșirea encoderului au 768 de dimensiuni, cele două reprezentări nu sunt direct comparabile doar pe baza dimensionalității. Prima aparține unui spațiu textual derivat din **UniXcoder**, în timp ce a doua aparține unui spațiu structural învățat pe graf. Din acest motiv, ieșirea structurală a encoderului nu este folosită direct pentru compararea cu in-

terogările reprezentate textual în spațiul menționat anterior. În schimb, ieșirea encoder-ului trece printr-o transformare suplimentară (două straturi liniare cu normalizare și activare), conform ideii din învățarea contrastivă că un *head* de proiecție poate fi folosit la antrenare pentru a îmbunătăți alinierea, în timp ce reprezentările finale rămân comparabile la inferență [22], care o aduc explicit în spațiul textual de 768 de dimensiuni folosit la antrenarea proiecției de interogări, realizată înainte de antrenare.

Prin urmare, la retrieval, reprezentarea fiecărui nod (encoder și transformarea finală) și reprezentarea fiecărei interogări sunt comparabile prin similaritate cosinus, deoarece au aceeași dimensiune și același scop, ce înseamnă potrivirea unei descrieri în limbaj natural cu funcția țintă din graf, în aceeași logică a alinierii contrastive între două modalități (text și noduri de cod), analogă paradigmei *CLIP* [23] (analogie doar).

Antrenarea este semisupervizată și contrastivă, fiecare interogare trebuie să fie mai similară nodului pozitiv asociat decât celorlalte noduri pozitive din lot, prin entropie încrucișată peste scoruri normalizate la temperatură fixă, adică *InfoNCE* cu negative în lot, regularizată prin *label smoothing* [24]. Obiectivul nu mai este discriminatorul bilinear din *DGI*, ci alinierea directă între fiecare interogare și nodul țintă (target node) asociat. În timpul antrenării, muchiile sunt eliminate aleator ocazional, ca augmentare structurală în spiritul *GraphCL* [25].

4.3.4 Căutarea semantică de clustere prin PR

Interfața paginii dedicate găsirii nodurilor pe baza interogărilor în limbaj natural este prezentată în Figura 2.3 din Anexa 2.

Căutarea semantică [5] este expusă în interfață printr-un panou lateral deschis din canvas-ul grafului de cunoștințe. Utilizatorul introduce o interogare în limbaj natural, iar înainte de inferență, sistemul verifică dacă proiectul are embedding-urile nodurilor preprocesate și dacă este necesar, execută etapa de vectorizare a codului. Apoi interfața solicită server-ului calculul cluster-ului, transmițând textul interogării și dimensiunea dorită a grupului de noduri.

Pe server, căutarea pornește de la similaritatea semantică în spațiul **UniXcoder** (interogare proiectată și embedding-uri brute) [8], apoi extinde rezultatul prin propagare pe graf, nu prin reprezentările *DGI* antrenate semisupervizat. Mai întâi, interogarea este proiectată în același spațiu de 768 de dimensiuni ca embedding-urile nodurilor, iar nodul cu cea mai mare similaritate cosinus devine nodul de ancorare (*seed*), adică funcția cu cea mai bună potrivire semantică, de la care pornește extinderea către vecinătate.

Apoi, pe graful structural, se aplică algoritmul *PageRank* personalizat (denumit la evaluarea extrinsecă drept PPR/ Personalized PR), derivat din *PageRank* [2], ce simulează o explorare repetată a grafului pornind de la nodul *seed*, presupunând că majoritatea pașilor, în procent de 85% (cauzat de factorul de amortizare de 0.85) urmează muchiile de la seed către funcții conectate, dar periodic

(15%) explorarea revine la punctul de start, nodul *seed*, astfel încât relevanța rămâne ancorată semantic, iar căutarea rămâne concentrată în jurul funcției de ancorare, fără să exploreze noduri funcții fără legătură directă. După identificarea nodului *seed* prin similaritate semantică, se aplică un *random walk with restart* pe graful de dependențe, iar la fiecare pas, explorarea continuă pentru restul muchiilor sau revine la *seed*. Astfel, funcțiile structural apropiate de *seed* primesc scoruri mai mari, chiar dacă textul lor nu seamănă direct cu interogarea.

Rezultatul cluster-ului selectat nu este format doar din top-ul scorurilor globale format prin *PR*, ci reprezintă un grup local și coerent pe structura codului. Inițial, cluster-ul cuprinde funcția de ancorare și vecinii săi direcți (la distanță unu pe graf/ *one hop neighbours*, adică funcțiile legate imediat prin apeluri sau dependențe).

Ulterior, dacă utilizatorul cere o dimensiune mai mare a grupului, lista se extinde prin metoda cotului (*elbow*). Metoda urmărește cum scad scorurile *PR* ale nodurilor următoare și se oprește acolo unde apare o scădere bruscă, adică unde funcțiile adăugate ar fi doar slab legate de subiect, spre deosebire de cele deja incluse. Rezultatul, o listă de identificatori de noduri, este trimis înapoi către interfață, care evidențiază cluster-ul pe canvas. Utilizatorul poate deselecta noduri irelevante și confirma un subset ca *seed* validat pentru pasul de generare al documentației.

4.3.5 Generarea documentației prin DGI

Interfața generării documentației care este ilustrată în Figura 2.4 din Anexa 2.

Documentația proiectului este evidențiată într-un panou lateral dedicat, accesibil din zona grafului de cunoștințe. La deschidere, interfața încarcă fie un draft editabil salvat anterior de utilizator, fie documentația de bază generată automat, dacă aceasta există deja.

Generarea pornește la cererea explicită a utilizatorului și presupune că există cel puțin un cluster validat în căutarea semantică (marcat ca păstrat pentru documentație). Astfel, server-ul pornește de la graful preprocesat al proiectului, calculează reprezentările nodurilor cu encoderul *DGI* semi-supervizat și aplică o *clusterizare aglomerativă ierarhică* [4] în două etape, întâi zone funcționale extinse, apoi subgrupuri mai strânse în interiorul zonelor care conțin suficiente funcții. În acest proces, funcțiile cu reprezentări *DGI* similare sunt grupate folosind distanța cosinus, însemnând că decizia de a uni sau de a separa grupuri se ia după distanța medie între ele. Apoi, în cadrul fiecărui cluster, funcțiile sunt clasificate pentru afișare în documentație după un scor în care proximitatea față de centrul semantic are ponderea principală, iar gradul de conectivitate în graf are un rol secundar (scor final format din procente de 70% proximitate semantică și 30% proximitate structurală).

În cazul în care există commit-uri analizate, este adăugată o secțiune despre evoluția temporală a modificărilor, prin gruparea commit-urilor după embedding-urile funcțiilor schimbate.

Ulterior, serverul îmbină documentația de bază cu clusterelor validate manual de utilizator în

căutarea *PR*, astfel încât pentru fiecare seed păstrat se includ interogarea originală și funcțiile selectate, alături de rezumatul clusterelor *DGI*. Se obține astfel un draft editabil, pe care utilizatorul îl poate modifica și salva ca versiune proprie.

Prin această abordare, documentația suprinde atât clusterelor curatoriate de utilizator (zonele de cod identificate prin căutare semantică cu *PR* și confirmate explicit), cât și structura descoperită automat prin embedding-urile semisupervizate *DGI* (clusterelor ierarhice pe tot proiectul).

4.3.6 Evaluarea metodelor abordate

Pentru a justifica metodele abordate, am evaluat encoder-ele și strategiile de recuperare pe corpus-ul de 67 de proiecte folosit la antrenare. Evaluările măsoară calitatea clusterizării, păstrarea vecinătății structurale și recuperarea clusterelor pe baza interogărilor.

4.3.6.1 Evaluarea recuperării de noduri în mod intrinsec

Prin recuperare (*retrieval*), se verifică dacă reprezentările vectoriale păstrează structura grafului. Spre deosebire de evaluările condiționate de limbaj natural, aici nu intervin interogările utilizatorului, se testează doar dacă funcțiile conectate direct în graf sunt apropiate și în spațiul de embedding.

Se compară atât embedding-urile **UniXcoder** brute ca reprezentare semantică cu cele produse de encoderul *DGI* semisupervizat ca reprezentare structurală, iar pentru fiecare nod care are cel puțin un vecin, se construiește vecinătatea reală la un salt (*one hop neighbours*), adică mulțimea funcțiilor legate direct prin apeluri sau dependențe. Cardinalitatea acestei mulțimi variază la fiecare nod, de aceea în spațiul de embedding, pentru același nod se recuperează un număr fix de 20 de candidați, cei mai apropiați după similaritate cosinus. Alegerea unei valori constante pentru toate nodurile permite o comparație uniformă între reprezentări, chiar dacă vecinătatea reală are mărimi diferite.

Într-un final, suprapunerea dintre nodurile recuperate și vecinii reali este măsurată prin scorurile de precizie, recall și F1. Pe media celor 67 de proiecte, **UniXcoder** brut obține precizie 0.102, recall 0.473 și F1 0.141, în timp ce *DGI* nesupervizat atinge 0.172, 0.725 și 0.233, iar *DGI* finetuned 0.167, 0.688 și 0.225. Ambele variante *DGI* depășesc **UniXcoder** la toate cele trei metrici, unde diferența față de **UniXcoder** brut este de peste 0.2 puncte la recall. *DGI* nesupervizat rămâne varianta ideală în context intrinsec, ce sugerează că alinierea la spațiul interogărilor în antrenarea semisupervizată nu îmbunătățește recuperarea nodurilor intrinsec. Rezultatele acestei evaluări sunt prezentate în Figura 1.1 și în Tabelul 1.1 din Anexa 1.

4.3.6.2 Evaluarea recuperării de noduri în mod extrinsec, condiționat de interogări

Această evaluare măsoară dacă sistemul bazat pe o interogare recuperează nodurile vecine relevante codului sursă din jurul nodului țintă. Experimentele rulează pe split-urile de validare și test ale perechilor interogare-nod, astfel încât interogările nu coincid cu cele folosite la antrenarea se-

misupervizată.

Ground truth-ul pentru fiecare interogare este nodul țintă împreună cu vecinii săi direcți din graf (vecinătatea la un salt, *one hop neighbours*), iar cardinalitatea set-ului de noduri variază în funcție de complexitatea interogării și de gradul nodului țintă.

Pentru o comparație relevantă între metode, dimensiunea setului recuperat nu este fixată global (spre deosebire de evaluarea intrinsecă, unde numărul de vecini e fixat la 20). Pentru fiecare interogare se calculează un *budget*, care e plafonul adaptiv al nodurilor returnate per interogare, derivat din vecinătatea seed-ului. În literatura de retrieval, *budget-ul* limitează resursele per interogare: documente returnate, comparații de distanță, tokeni ș.a., iar în cazul curent resursa limitată e numărul de noduri din răspuns. În continuare, se proiectează interogarea în spațiul **UniXcoder**, se identifică nodul *seed* de ancorare semantică (cel mai similar), apoi numărul căutat devine dimensiunea vecinătății la un salt a acestui nod. Același *budget* (k) este impus tuturor metodelor care selectează un set de cardinalitate fixă (în principal variantele bazate pe k -NN, plus *dgi_seed_ppr*, care trunchiază scorurile PR la top- k), astfel încât diferențele să reflecte strategia de selecție, nu dimensiunile diferite ale setului returnat.

Așadar, se compară mai multe strategii, printre care căutare k -NN în spațiul brut sau proiectat, căutare directă în spațiul *DGI*, reranking al unui pool de candidați cu reprezentări *DGI*, propagare structurală cu ancorare semantică în spațiul brut (variantele din interfață), propagare cu *seed* ales în spațiul *DGI* și alte combinații hibride. Metodele bazate pe *PR* pot returna un set de dimensiune adaptivă (vecinătate inițială plus extindere prin metoda *elbow*).

Se raportează precizia, recall-ul și F1-ul ca suprapunere între setul recuperat și ground truth-ul la un salt (*one hop*), MRR și Hits@ k pentru metodele care produc un ranking explicit, precum și similaritatea semantică medie a nodurilor recuperate față de interogare.

Rezultatul principal este că *raw_seed_ppr*, metoda din interfață, domină recuperarea *one hop* (F1 0.758, recall și precizie 0.765), depășind atât k -NN brut/ proiectat (F1 0.392/ 0.387), cât și variantele bazate pe *DGI* (F1 sub 0.23). Aceasta confirmă un contrast important față de evaluarea intrinsecă: embedding-urile *DGI* păstrează mai bine vecinătatea structurală fără interogări, dar nu se traduc în performanță superioară la căutare în limbaj natural.

La ranking, *raw_knn* și *proj_knn* sunt superioare (MRR 0.750/ 0.757), în timp ce *PPR* rămâne mai bun la suprapunerea setului recuperat decât la poziționarea țintei în listă. Similaritatea semantică maximă la *raw_knn* (0.444) nu implică F1 mare, deoarece ground truth-ul *one hop* include și vecini structurali puțin similari textual cu interogarea, deci *PPR* compensează prin propagare pe graf.

Rezultatele comparative pentru această evaluare sunt prezentate în Figura 1.2 și în Tabelele 1.2, 1.3 și 1.4 din Anexa 1.

4.3.6.3 Evaluarea clusterizării

Se compară embedding-urile produse de encoderele *DGI* (nesupervizat și semisupervizat) cu reprezentări de referință obținute prin **CodeBERT** [7] și **Node2Vec** [26]. Pentru fiecare proiect se aplică aceeași schemă de *clusterizare aglomerativă* folosită la documentație și se raportează calitatea grupărilor prin scorul Silhouette, indicele Davies–Bouldin și puritatea. *Silhouette* [27] și indicele *Davies–Bouldin* [28] provin din literatura clusterizării și măsoară compactitatea și separarea grupărilor în spațiul embedding-urilor, independent de etichete externe, în timp ce puritatea [29] este o măsură de validare externă, utilizată frecvent când ground truth-ul lipsește sau este incomplet, care sugerează cât de omogene sunt clusterele după locația în proiect: urmărește ideea ca funcțiile din același cluster să provină din același modul/ folder. În această lucrare, metricile sunt aplicate uniform tuturor reprezentărilor (*DGI*, *CodeBERT*, *Node2Vec*), astfel încât diferențele sugerează calitatea embedding-urilor.

DGI nesupervizat obține cel mai mare scor Silhouette mediu (0.566) și cel mai mic indice Davies–Bouldin (0.741), ceea ce indică clusterare mai bine separate și mai compacte decât la *DGI* semisupervizat și **Node2Vec**. *DGI* semisupervizat atinge cea mai mare puritate (0.931), ușor peste varianta nesupervizată (0.911), astfel că grupările rămân aliniate la prefixele de folder/ modul derivate din calea fișierelor. În final, varianta semisupervizată este aleasă deoarece este adecvată pentru sarcinile legate de interogări, obiectivul ei fiind alinierea interogare-nod, nu maximizarea separării clusterelor, aspect pe care scorurile Silhouette și Davies–Bouldin nu îl capturează singure. Compararea acestor rezultate este prezentată în Figura 1.3 și în Tabelul 1.5 din Anexa 1.

4.4 Partajarea proiectelor între utilizatori

4.4.1 Trimiterea și primirea pachetelor de date

Partajarea unui proiect este inițiată din interfață de utilizatorul care deține proiectul, unde alege destinatarul după nume de utilizator sau adresă de e-mail, specificând un mesaj și rolul acordat. Server-ul împachetează într-o arhivă ZIP datele locale ale proiectului, mai exact graful structural, istoricul commit-urilor, documentația și rezultatele analizelor stocate în **AppData**, împreună cu un manifest care descrie conținutul export-ului. Arhiva este încărcată în PocketBase, unde se creează o înregistrare de partajare cu starea **pending**, asociată proiectului, expeditorului și destinatarului.

Destinatarul vede invitațiile în aplicație și poate accepta sau refuza importul. La acceptare, arhiva este extrasă în spațiul local al destinatarului, iar căile de citire/ scriere din graful de cunoștințe sunt adaptate la mediul nou, astfel încât analiza să poată continua fără dependență de structura de directoare a expeditorului. Altfel, refuzul destinatarului actualizează doar starea înregistrării, fără a transfera date. Modelul este asincron, însemnând că actualizările ulterioare necesită o nouă partajare din partea proprietarului.

4.4.2 Roluri, acces și notificări

Accesul la un proiect partajat se bazează pe roluri. Proprietarul păstrează controlul complet asupra configurării GitHub, partajării și ștergerii proiectului. Destinatarul cu rol de vizualizator (**viewer**) poate consulta graful, documentația și istoricul commit-urilor, configura folderul sursă local și rerula analiza structurală, dar nu poate modifica documentația generată. Rolul de **editor** prezintă posibilitatea de a edita documentația și rezultatele de clustering. Pentru ambele roluri, evenimentele GitHub și sincronizarea commit-urilor păstrează integrarea configurată de proprietar, astfel încât toți utilizatorii alături de care a fost partajat un proiect să lucreze în același repository.

În plus, expeditorul este notificat când un destinatar acceptă sau refuză un pachet, iar destinatarul este informat dacă proprietarul șterge proiectul, moment în care accesul este revocat.

5 Concluzie

Aplicația reprezintă un instrument, care se bazează pe o abordare structurală, semantică și temporală a codului sursă, oferind atât o perspectivă vizuală reprezentativă pentru navigarea într-un proiect, cât și sugestii în vederea generării unei documentații referitoare la proiect. Abordarea acestei lucrări îmbină analiza statică a codului sursă, integrarea cu GitHub pentru urmărirea evoluției în timp, împreună cu reprezentări învățate ale grafului de cunoștințe și mecanisme de căutare pe baza interogărilor în limbaj natural, într-o arhitectură local-first, în care datele analitice rămân pe dispozitivul utilizatorului, iar metadatele de acces și partajare sunt gestionate centralizat.

Ca direcții de dezvoltare ulterioară, e necesar suportul pentru alte recunoașterea altor limbaje de programare, consolidarea corespondenței dintre embedding-urile *DGI* și interogările în limbaj natural, păstrând în același timp calitatea recuperării structurale, precum și îmbunătățirea fluxurilor de colaborare între deținători și destinatari ai proiectelor.

În plus, rezultatele obținute sunt relevante deoarece confirmă o diferență utilă în practică, mai exact că reprezentările învățate nu trebuie evaluate doar printr-o singură metrică universală, ci în funcție de sarcina pentru care sunt folosite. Evaluarea intrinsecă arată că *DGI* păstrează mai bine structura grafică decât embedding-urile statice, în schimb, evaluarea extrinsecă demonstrează că interogările în limbaj natural beneficiază de o ancorare semantică în spațiul brut, completată apoi de relațiile din cod.

Bibliografie

- [1] Alfred V. Aho, Monica S. Lam, Ravi Sethi, and Jeffrey D. Ullman. *Compilers: Principles, Techniques, and Tools*. Addison-Wesley, 2 edition, 2006.
- [2] Lawrence Page, Sergey Brin, Rajeev Motwani, and Terry Winograd. The PageRank citation ranking: Bringing order to the web. Technical report, Stanford InfoLab, 1999.
- [3] Petar Veličković, William Fedus, William L. Hamilton, Pietro Liò, Yoshua Bengio, and R. Devon Hjelm. Deep graph infomax. In *International Conference on Learning Representations*, 2019. <https://arxiv.org/abs/1809.10341>.
- [4] Joe H. Ward. Hierarchical grouping to optimize an objective function. *Journal of the American Statistical Association*, 58(301):236–244, 1963.
- [5] Hamel Husain, Ho-Hsiang Wu, Tiferet Gazit, Miltiadis Allamanis, and Marc Brockschmidt. CodeSearchNet challenge: Evaluating the state of semantic code search. In *NeurIPS ML4Code Workshop*, 2019. <https://arxiv.org/abs/1909.09436>.
- [6] Thomas N. Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations*, 2017. <https://arxiv.org/abs/1609.02907>.
- [7] Zhangyin Feng, Daya Guo, Duyu Tang, et al. CodeBERT: A pre-trained model for programming and natural languages. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, 2020. <https://aclanthology.org/2020.findings-emnlp.139>.
- [8] Daya Guo, Shuai Lu, Nan Duan, Yanlin Wang, Ming Zhou, and Jian Yin. UniXcoder: Unified cross-modal pre-training for code representation. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics*, pages 7212–7225, 2022. <https://aclanthology.org/2022.acl-long.499>.
- [9] Sourcegraph. Code understanding, oversight and evolution. <https://sourcegraph.com>, accesat: 07.06.2026.
- [10] Sonde Developer Tools. Sonde: Structural context for high-velocity code. <https://www.getsonde.com>, accesat: 07.06.2026.
- [11] Pieces for Developers. Pieces desktop application. <https://docs.pieces.app/products/desktop>, accesat: 07.06.2026.

- [12] Anna Abad Sieper, Omar Amarkhel, Savina Diez, and Dominic Petrak. Semantic code search with neural bag-of-words and graph convolutional networks. In *SKILL 2020, Lecture Notes in Informatics (LNI) – Seminars*, Bonn, 2020. Gesellschaft für Informatik. <https://dl.gi.de/server/api/core/bitstreams/429a559b-4275-4ac6-9a9b-f40ffd886594/content>.
- [13] Martin Kleppmann, Adam Wiggins, Peter van Hardenberg, and Mark McGranaghan. Local-first software: You own your data, in spite of the cloud. In *Onward! 2019*, pages 154–178. ACM, 2019.
- [14] Good Code. Reagraph: High-performance network graph visualization for React. [url{https://reagraph.dev/docs}](https://reagraph.dev/docs), accesat: 07.06.2026.
- [15] David Sherret. ts-morph: TypeScript compiler API wrapper for static analysis and programmatic code changes, 2025. <https://github.com/dsherret/ts-morph>, accesat: 07.06.2026.
- [16] Adam Paszke, Sam Gross, Francisco Massa, et al. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, volume 32, 2019. https://papers.neurips.cc/paper_files/paper/2019/file/bdbca288fee7f92f2bfa9f7012727740-Paper.pdf.
- [17] Matthias Fey and Jan Eric Lenssen. Fast graph representation learning with PyTorch Geometric. In *ICLR Workshop on Representation Learning on Graphs and Manifolds*, 2019. <https://arxiv.org/abs/1903.02428>.
- [18] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, et al. Scikit-learn: Machine learning in python. *Journal of Machine Learning Research*, 12:2825–2830, 2011. <https://www.jmlr.org/papers/v12/pedregosa11a.html>.
- [19] William L. Hamilton, Rex Ying, and Jure Leskovec. Inductive representation learning on large graphs. In *Advances in Neural Information Processing Systems*, 2017. <https://arxiv.org/abs/1706.02216>.
- [20] Yue Wang, Weishi Wang, Shafiq Joty, and Steven C. H. Hoi. CodeT5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, 2021. <https://aclanthology.org/2021.emnlp-main.685>.

- [21] Aaron van den Oord, Yazhe Li, and Oriol Vinyals. Representation learning with contrastive predictive coding. arXiv preprint arXiv:1807.03748, 2018. <https://arxiv.org/abs/1807.03748>.
- [22] Ting Chen, Simon Kornblith, Mohammad Norouzi, and Geoffrey Hinton. A simple framework for contrastive learning of visual representations. In *International Conference on Machine Learning*, pages 1597–1607, 2020. <https://proceedings.mlr.press/v119/chen20j.html>.
- [23] Alec Radford, Jong Wook Kim, Chris Hallacy, et al. Learning transferable visual models from natural language supervision. In *International Conference on Machine Learning*, 2021. <https://proceedings.mlr.press/v139/radford21a.html>.
- [24] Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jon Shlens, and Zbigniew Wojna. Rethinking the inception architecture for computer vision. In *IEEE Conference on Computer Vision and Pattern Recognition*, pages 2818–2826, 2016. <https://arxiv.org/abs/1512.00567>.
- [25] Yuning You, Tianlong Chen, Yongduo Sui, Ting Chen, Zhangyang Wang, and Yang Shen. Graph contrastive learning with augmentations. In *Advances in Neural Information Processing Systems*, volume 33, 2020. <https://arxiv.org/abs/2010.13902>.
- [26] Aditya Grover and Jure Leskovec. node2vec: Scalable feature learning for networks. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 855–864, 2016. <https://arxiv.org/abs/1607.00653>.
- [27] Peter J. Rousseeuw. Silhouettes: A graphical aid to the interpretation and validation of cluster analysis. *Journal of Computational and Applied Mathematics*, 20:53–65, 1987. https://www.researchgate.net/publication/340575202_Silhouettes_A_graphical_aid_to_the_interpretation_and_validation_of_cluster_analysis.
- [28] David L. Davies and Donald W. Bouldin. A cluster separation measure. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, PAMI-1(2):224–227, 1979. https://www.researchgate.net/publication/224377470_A_Cluster_Separation_Measure.
- [29] Christopher D. Manning, Prabhakar Raghavan, and Hinrich Schütze. *Introduction to Information Retrieval*. Cambridge University Press, Cambridge, UK, 2008. <https://nlp.stanford.edu/IR-book/pdf/irbookonlinereading.pdf>.

Anexă 1: Rezultatele evaluărilor experimentale

Această anexă include figurile asociate evaluărilor experimentale discutate în secțiunea **Evaluarea metodelor abordate**.

Evaluarea recuperării de noduri în mod intrinsec

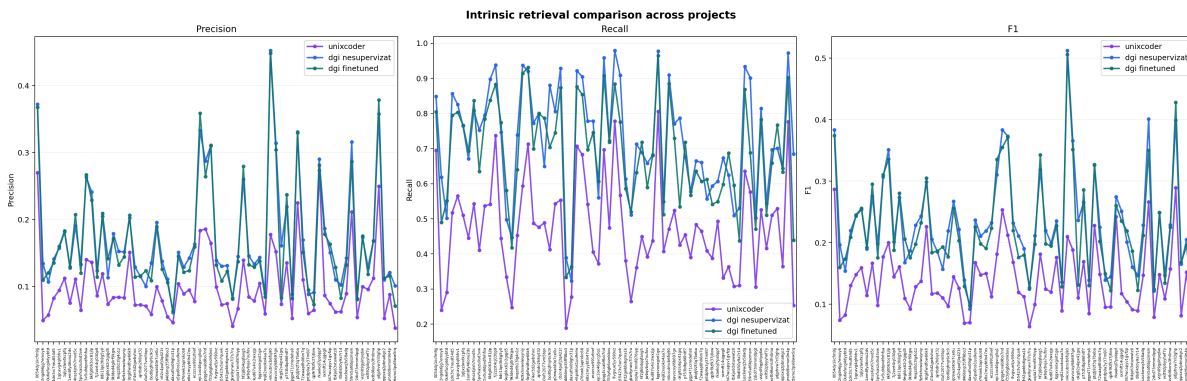


Figura 1.1: Evaluarea intrinsecă a recuperării de noduri prin compararea vecinătății structurale cu nodurile apropiate în spațiul de embedding

Tabela 1.1: Evaluarea intrinsecă: media metricilor de recuperare la nivel de proiecte

Metodă	Precision	Recall	F1
UniXcoder	0.102	0.472	0.140
DGI nesupervizat	0.172	0.724	0.233
DGI finetuned	0.167	0.688	0.224

Evaluarea recuperării de noduri în mod extrinsec

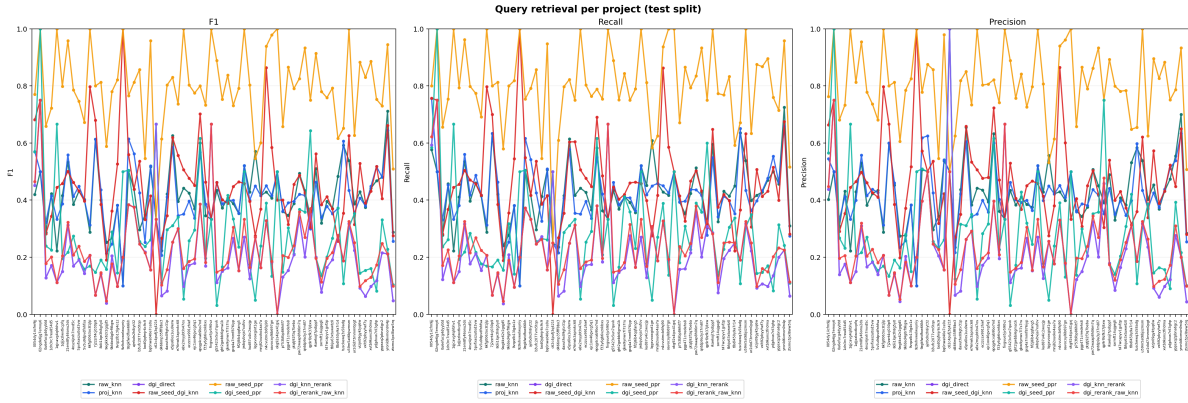


Figura 1.2: Evaluarea extrinsecă a recuperării de noduri pe split-ul de test pentru metodele condiționate de interogări în limbaj natural

Tabela 1.2: Evaluarea extrinsecă: metrici de recuperare pentru vecinătatea la un salt

Metodă	F1	Recall	Precision
raw_knn	0.392 $\pm$ 0.232	0.406 $\pm$ 0.246	0.401 $\pm$ 0.240
proj_knn	0.387 $\pm$ 0.231	0.403 $\pm$ 0.246	0.394 $\pm$ 0.235
dgi_direct	0.163 $\pm$ 0.248	0.173 $\pm$ 0.264	0.168 $\pm$ 0.258
raw_seed_dgi_knn	0.415 $\pm$ 0.294	0.423 $\pm$ 0.297	0.424 $\pm$ 0.306
raw_seed_ppr	0.758 $\pm$ 0.388	0.765 $\pm$ 0.387	0.765 $\pm$ 0.386
dgi_seed_ppr	0.226 $\pm$ 0.328	0.233 $\pm$ 0.338	0.234 $\pm$ 0.339
dgi_knn_rerank	0.163 $\pm$ 0.248	0.173 $\pm$ 0.264	0.168 $\pm$ 0.258
dgi_rerank_raw_knn	0.192 $\pm$ 0.252	0.204 $\pm$ 0.270	0.197 $\pm$ 0.262

Tabela 1.3: Evaluarea extrinsecă: metrici de ranking MRR și Hits@k

Metodă	MRR	Hits@1	Hits@5	Hits@10
raw_knn	0.750 $\pm$ 0.361	0.646	0.882	0.894
proj_knn	0.757 $\pm$ 0.363	0.663	0.880	0.889
dgi_direct	0.140 $\pm$ 0.296	0.086	0.204	0.246
raw_seed_dgi_knn	0.694 $\pm$ 0.439	0.661	0.738	0.757
raw_seed_ppr	0.616 $\pm$ 0.421	0.504	0.742	0.756
dgi_seed_ppr	0.112 $\pm$ 0.272	0.070	0.166	0.189
dgi_knn_rerank	0.140 $\pm$ 0.296	0.086	0.204	0.246
dgi_rerank_raw_knn	0.175 $\pm$ 0.315	0.101	0.272	0.316

Tabela 1.4: Evaluarea extrinsecă: relevanța semantică medie față de interogare, calculată prin similaritate cosinus în spațiul UniXcoder brut

Metodă	Medie $\pm$ deviație standard
raw_knn	0.444 $\pm$ 0.086
proj_knn	0.427 $\pm$ 0.092
dgi_direct	0.276 $\pm$ 0.101
raw_seed_dgi_knn	0.349 $\pm$ 0.103
raw_seed_ppr	0.345 $\pm$ 0.100
dgi_seed_ppr	0.253 $\pm$ 0.102
dgi_knn_rerank	0.276 $\pm$ 0.101
dgi_rerank_raw_knn	0.325 $\pm$ 0.087

Evaluarea clusterizării

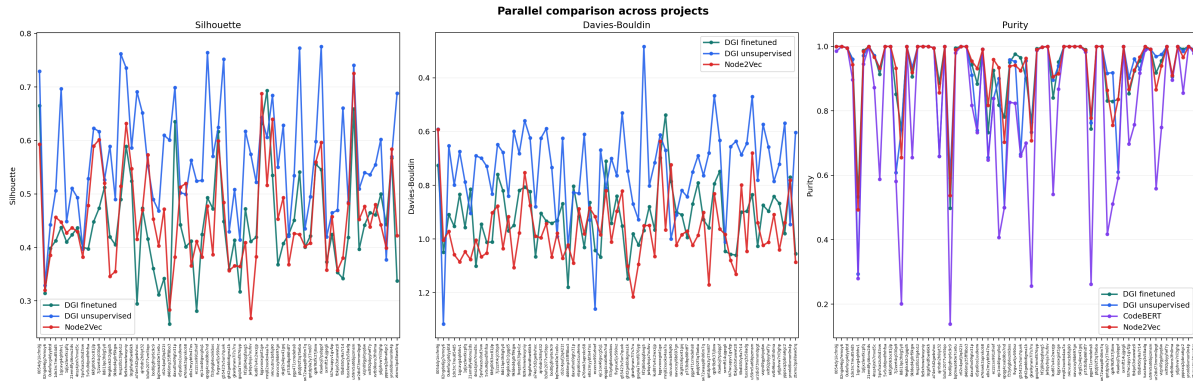


Figura 1.3: Evaluarea clusterizării prin compararea reprezentărilor analizate în raport cu metricile Silhouette, Davies-Bouldin și puritate

Tabela 1.5: Evaluarea clusterizării

Metodă	Silhouette	Davies-Bouldin	Puritate
DGI nesupervizat	0.566 ± 0.107	0.741 ± 0.170	0.911 ± 0.147
DGI finetuned	0.445 ± 0.094	0.914 ± 0.116	0.931 ± 0.106
Node2Vec	0.457 ± 0.095	0.967 ± 0.121	0.932 ± 0.109
CodeBERT	necalculat	necalculat	0.814 ± 0.241

Anexă 2: Interfața aplicației

Această anexă include capturi din interfața aplicației, folosite pentru a ilustra componentele funcționale discutate în lucrare.

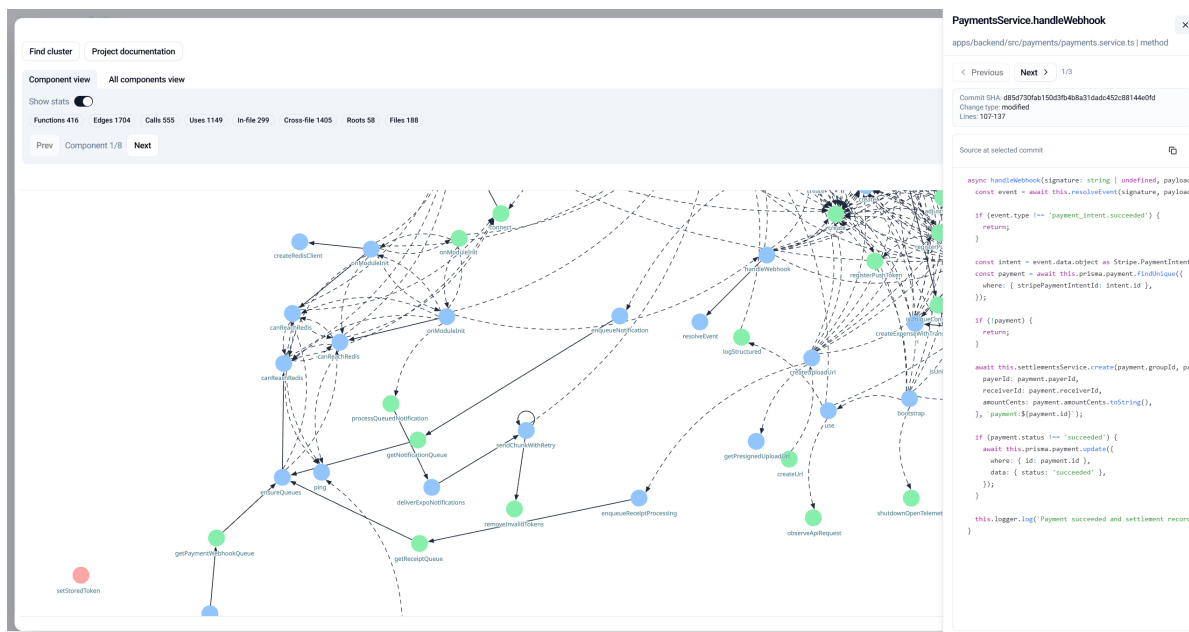


Figura 2.1: Interfața aplicației pentru canvasul grafului de cunoștințe, unde nodurile și muchiile rezultate din analiza statică sunt afișate pentru explorarea structurii proiectului

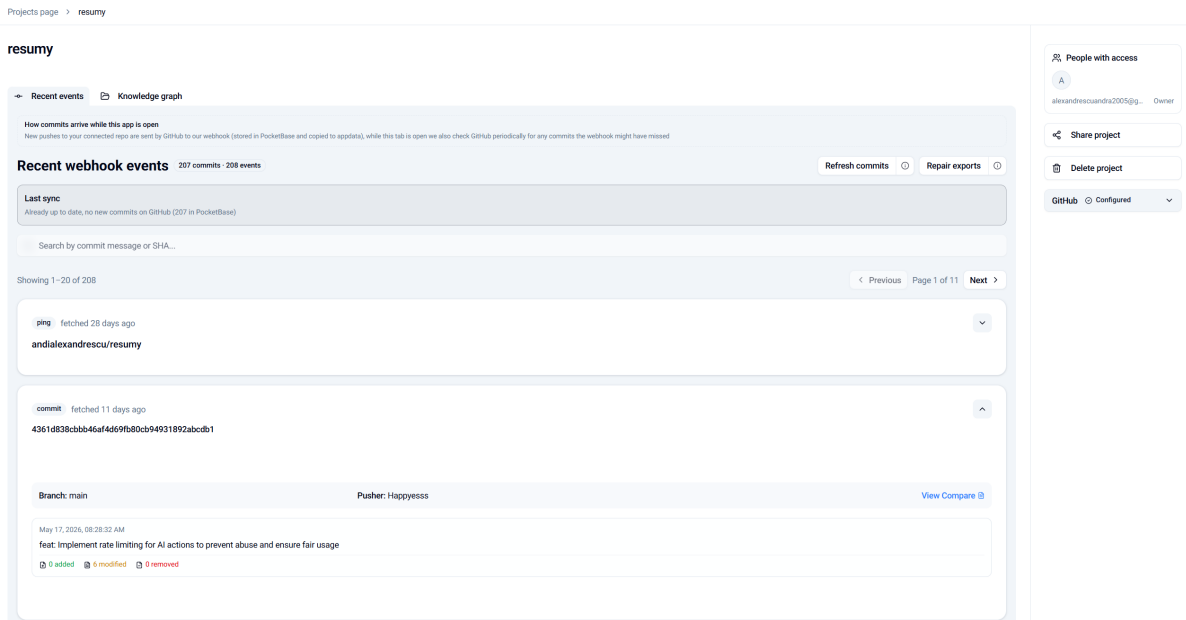


Figura 2.2: Interfața aplicației pentru pagina evenimentelor webhook, unde sunt afișate evenimentele GitHub recepționate și sincronizate de sistem

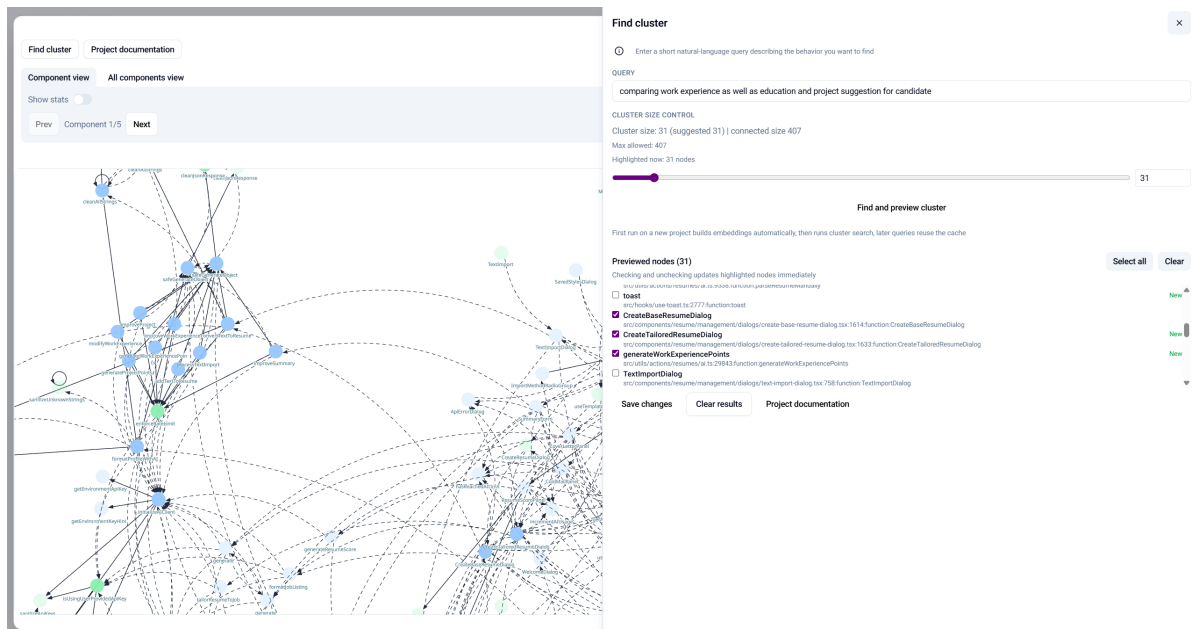


Figura 2.3: Interfața aplicației pentru căutarea semantică de clustere prin PR, unde utilizatorul formulează interogarea și inspectează rezultatele relevante din graful de cunoștințe

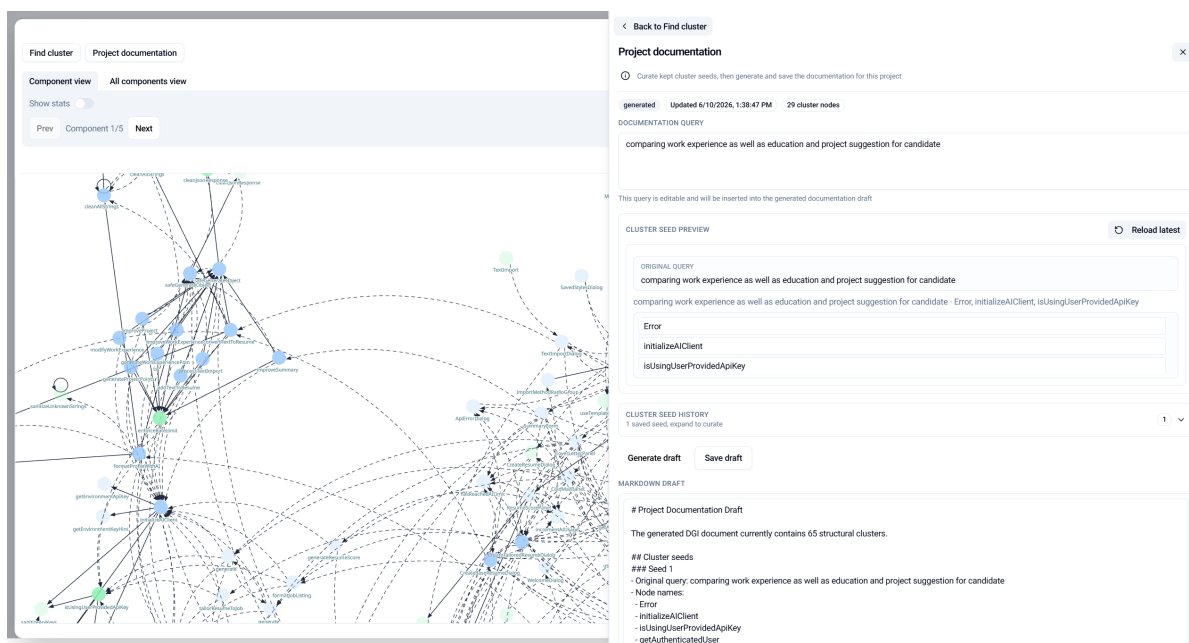


Figura 2.4: Interfața aplicației pentru documentația proiectului, unde draft-ul structural generat pe baza clusterelor poate fi inspectat și editat de utilizator